Nama : Wanda Pramudya R Matkul : Pemograman Web 2

Kelas : 1243B Dosen : Agung Nugroho, S.Kom., M.Kom.

Nim : 312410102 Tugas : Ulangan Tengah Semester

Belajar SQL Injection Lewat Percobaan Login Aplikasi Web Sederhana

Pendahuluan

Perkembangan aplikasi web saat ini semakin pesat dan hampir semua layanan digital terhubung langsung dengan database. Mulai dari sistem login, penyimpanan data pengguna, hingga transaksi online, semuanya bergantung pada keamanan pengelolaan data. Namun, masih banyak aplikasi web yang dibuat tanpa memperhatikan aspek keamanan secara serius.

Salah satu celah keamanan yang cukup sering ditemukan adalah **SQL Injection**. Serangan ini terjadi karena aplikasi web tidak memvalidasi input pengguna dengan benar. Melalui artikel ini, saya mencoba mempelajari SQL Injection tidak hanya dari teori, tetapi juga melalui eksperimen langsung menggunakan aplikasi login sederhana untuk memahami bagaimana serangan ini bekerja dan bagaimana cara mencegahnya.

Pengertian SQL Injection

SQL Injection merupakan teknik serangan yang memanfaatkan kelemahan aplikasi web ketika perintah SQL dibentuk langsung dari input pengguna. Jika input tersebut tidak diamankan, penyerang dapat menyisipkan perintah SQL tambahan yang mengubah logika query.

Celah ini sering ditemukan pada fitur login, form pencarian, maupun halaman lain yang berinteraksi langsung dengan database.

Gambaran Cara Kerja SQL Injection

Pada aplikasi login yang tidak aman, biasanya query SQL ditulis dengan cara menggabungkan input pengguna secara langsung. Contoh query yang rentan adalah sebagai berikut:

```
SELECT * FROM users
WHERE username =
'$username'
AND password = '$password';
```

Jika nilai $username dan $password berasal dari input pengguna tanpa validasi, maka query ini dapat dimanipulasi dengan mudah.

Eksperimen SQL Injection yang Saya Lakukan

Lingkungan Pengujian

Eksperimen ini saya lakukan menggunakan:

- Sistem Operasi: Windows
- Web Server: XAMPP
- Bahasa Pemrograman: PHP
- Database: MySQL
- Browser: Google Chrome

Saya membuat sebuah aplikasi login sederhana dengan tabel users yang berisi kolom id, username, dan password.

Pengujian Login Normal

Langkah pertama adalah mencoba login menggunakan data yang benar, yaitu:

- Username: admin
- Password: admin123

Hasilnya, sistem berhasil login seperti yang diharapkan. Ini menandakan bahwa fungsi login berjalan normal.

Percobaan SQL Injection

Setelah itu, saya mencoba memasukkan input tidak wajar pada kolom username, yaitu:

```
' OR '1'='1
```

Kolom password dibiarkan kosong.

Dengan input tersebut, query SQL yang dijalankan oleh sistem berubah menjadi:

```
SELECT * FROM users
WHERE username = '' OR '1'='1'
AND password = '';
SELECT * FROM users
WHERE username = '' OR '1'='1'
AND password = '';
```

Karena kondisi '1'='1' selalu bernilai benar, maka sistem tetap mengembalikan data dari database. Akibatnya, saya berhasil masuk ke dalam sistem tanpa memasukkan username dan password yang valid.

Hasil yang Didapat

Dari percobaan tersebut, saya menyimpulkan bahwa:

- Sistem login dapat dilewati dengan sangat mudah
- SQL Injection dapat memberikan akses ilegal ke dalam aplikasi
- Aplikasi tanpa pengamanan input sangat berisiko digunakan

Eksperimen ini menunjukkan bahwa celah kecil dalam penulisan query dapat berdampak besar terhadap keamanan sistem.

Dampak SQL Injection

Jika SQL Injection terjadi pada aplikasi nyata, dampaknya bisa sangat berbahaya, antara lain:

- Kebocoran data pengguna
- Perubahan atau penghapusan data penting
- Pengambilalihan akun administrator
- Kerusakan sistem dan penurunan kepercayaan pengguna

Oleh karena itu, serangan ini tidak boleh dianggap sepele.

Cara Mencegah SQL Injection

Menggunakan Prepared Statement

Salah satu cara paling efektif untuk mencegah SQL Injection adalah dengan menggunakan prepared statement. Teknik ini memisahkan perintah SQL dengan data input pengguna.

Contoh penerapan prepared statement pada PHP:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND
password=?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();
```

Dengan cara ini, input pengguna tidak akan diproses sebagai perintah SQL.

Validasi Input Pengguna

Selain prepared statement, setiap input pengguna juga perlu divalidasi dan disesuaikan dengan format yang diharapkan agar tidak mengandung karakter berbahaya.

Pembatasan Hak Akses Database

Akun database sebaiknya hanya diberikan hak akses sesuai kebutuhan aplikasi. Hal ini bertujuan untuk meminimalkan dampak jika serangan berhasil dilakukan.

Kesimpulan

Berdasarkan eksperimen yang saya lakukan, SQL Injection terbukti merupakan celah keamanan yang sangat berbahaya jika tidak ditangani dengan baik. Aplikasi login sederhana saja dapat dengan mudah ditembus jika input pengguna tidak diamankan.

Melalui penggunaan prepared statement, validasi input, dan pengaturan hak akses database, risiko SQL Injection dapat dikurangi secara signifikan. Eksperimen ini membuat saya lebih memahami pentingnya keamanan dalam pengembangan aplikasi web.

Referensi

- OWASP Foundation – SQL Injection
- Dokumentasi PHP – Prepared Statements
- W3Schools – SQL Injection

Link Artikel :
https://www.kompasiana.com/wandapramudya1401/69edf18aed64156e55678ce3/belajar-sql-injection-lewat-percobaan-login-aplikasi-web-sederhana?page=2&page_images=1

https://www.kompasiana.com/wandapramudya1401/69edf18aed64156e55678ce3/belajar-sql-injection-lewat-percobaan-log

kompasiana Beyond Blogging | Program | Terpopuler | Terbaru | Headline | Topik Pilihan | Komu

Ramadan New | Sustainability | Lyfe | Money | Vox Pop | Travel Story | Halo Lokal | Sports

wandapramudya

Karyawan

Hiking

KAMPUSIANA

Belajar SQL Injection Lewat Percobaan Login Aplikasi Web Sederhana

26 April 2026 18:05 | Diperbarui: 26 April 2026 18:05 | 10 0 0

Laporkan Konten